

Cahier des charges : **WhaWeb**

Front-end :

- **React.js (via Vite)** : Pour créer une application mono-page (SPA) rapide.
- **Tailwind CSS** : Pour le design et le style (gratuit et sans dépendances lourdes).
- **Lucide Icons** : Bibliothèque d'icônes open-source.

Back-end (Serveur & Logique) :

- **Node.js & Express.js** : Pour créer notre propre API (RESTful) qui fera le lien entre le Front-end et WhatsApp.

Base de données :

- **MongoDB Atlas** : Base de données NoSQL. largement suffisant pour stocker les contacts, les campagnes et les statistiques.

2. Intégration WhatsApp (L'API Gratuite)

- **Bibliothèque whatsapp-web.js (Node.js)** : Cette bibliothèque open-source utilise Puppeteer pour ouvrir un navigateur invisible en arrière-plan et se connecter à WhatsApp Web.
- **Authentification** : L'utilisateur scannera un QR Code affiché sur notre Dashboard (Front-end) pour lier son téléphone personnel ou professionnel à l'application.

3. Fonctionnalités & Provenance des Données

Dashboard :

A. Gestion des Contacts

Fonctionnalité : L'utilisateur peut ajouter des numéros de téléphone.

Provenance : Les contacts seront saisis sur le Front-end, envoyés à notre API Node.js, et sauvegardés dans notre base MongoDB.

B. Envoi de Campagnes (New Campaign)

Fonctionnalité : Créer un message et l'envoyer à une liste de contacts.

Le Processus :

1. Le Front-end envoie l'ordre au Back-end.

2. Le Back-end (via whatsapp-web.js) envoie les messages un par un aux numéros.

3. Le Back-end enregistre dans MongoDB l'historique : "Message envoyé à X à telle heure".

C. Statistiques & Dashboard

Les Chiffres (Messages Sent, Active Flows...) :

Provenance : Ne proviennent pas de WhatsApp directement, mais de notre base MongoDB. Notre API Node.js comptera combien de messages ont le statut "Envoyé" dans la base de données et renverra ce chiffre au Front-end.

Taux d'ouverture et de réponse :

Provenance : whatsapp-web.js permet d'écouter les événements (ex: message_ack pour savoir si le message a les deux traits bleus, ou message pour détecter une réponse). Notre Back-end mettra à jour ces statuts dans MongoDB pour mettre à jour les graphiques en temps réel.

4. Automatisation & Intelligence Artificielle (Optionnel - Avancé)

Pour transformer l'application en un véritable assistant intelligent, nous intégrerons une couche d'automatisation des réponses.

A. Orchestration via n8n

- Rôle : Utilisation de n8n (version self-hosted gratuite) comme moteur de workflow.
- Fonctionnement : Lorsqu'un message entrant est détecté par le Back-end (via whatsapp-web.js), il est envoyé à n8n via un Webhook.
- Avantage : Permet de créer des scénarios complexes (si le client demande un prix, envoyer le catalogue ; s'il demande un rendez-vous, vérifier l'agenda) sans coder chaque règle à la main.

B. Intelligence Artificielle Générative (LLM)

- Moteur IA : Intégration de l'API OpenAI (ChatGPT) ou d'un modèle Open-source comme Llama 3.
- Personnalisation (System Prompt) : L'utilisateur pourra configurer le "comportement" de l'IA depuis le Dashboard (ex: "Tu es un assistant commercial pour une agence de voyage...").
- Analyse de Sentiment : L'IA analysera le message du client pour comprendre son intention et générer une réponse fluide et naturelle, dépassant les capacités d'un simple

chatbot classique.

C. Interface de Configuration (AI Settings)

- Fonctionnalité : Ajout d'un onglet "AI Config" dans le Dashboard React.
- Contenu : Un champ de texte pour définir les instructions de l'IA (Knowledge Base) et un bouton pour activer/désactiver les réponses automatiques.